Generalized Linear Models

Tomáš Kysela

Introduction The aim of this assignment is to practice constructing linear models. You will start with a simple linear model. You will evaluate and interpret it (1p). Consequently, your task will be to improve this model using generalized linear models (GLMs) and feature transformations. You will get 1p for proposal and evaluation of GLM (family, evaluation, interpretation), 1p for correct feature transformations, 1p for proposal and justification of the final model and eventually, 1p for comprehensive evaluation of all the model improvements (ablation study through cross-validation, note that the previous evaluations must be done without cross-validation).

Input data In this assignment, you will work with a student dataset. The dataset contains 200 samples and 4 features: *num_awards* is the outcome variable and indicates the number of awards earned by students in a year, *math* is a continuous predictor variable and represents students' scores on their math final exam, *prog* is a categorical predictor variable with three levels indicating the type of program in which the students were enrolled (1 = "General", 2 = "Academic" and 3 = "Vocational"), and *work* is a continuous predictor that gives the number of hours that students spent at work on average per week.

```r
library(dplyr)
```

Load the necessary libraries and the dataset

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```r
library(caret) # comprehensive model evaluation
```

```
## Loading required package: ggplot2

## Loading required package: lattice
```

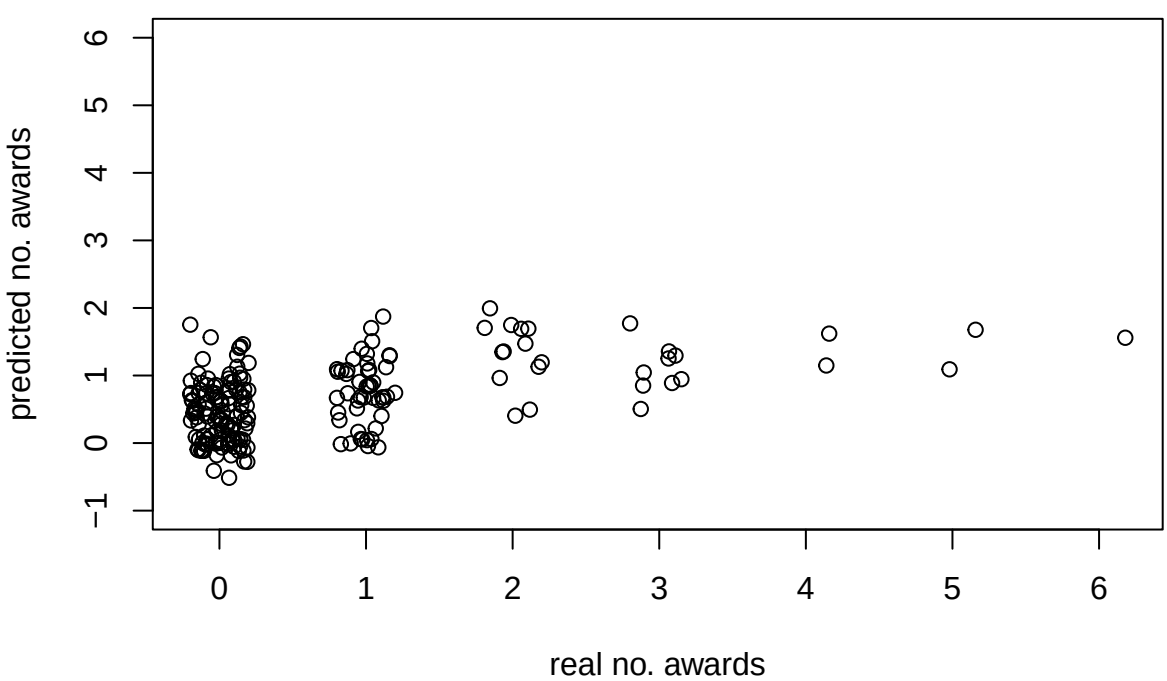
```r
library(splines)
library(ggplot2) # visualizations
library(rcompanion) # comparison of GLMs

d <- read.csv("study_data.csv")
```

Simple linear model Let us start with an ordinary linear model with no feature transformations. Explain how far the model works (does it meet formal assumptions?, does it overcome the null model?). Which predictors would you keep there and which of them are not useful? Use the standard evaluation procedures that we have for linear models.

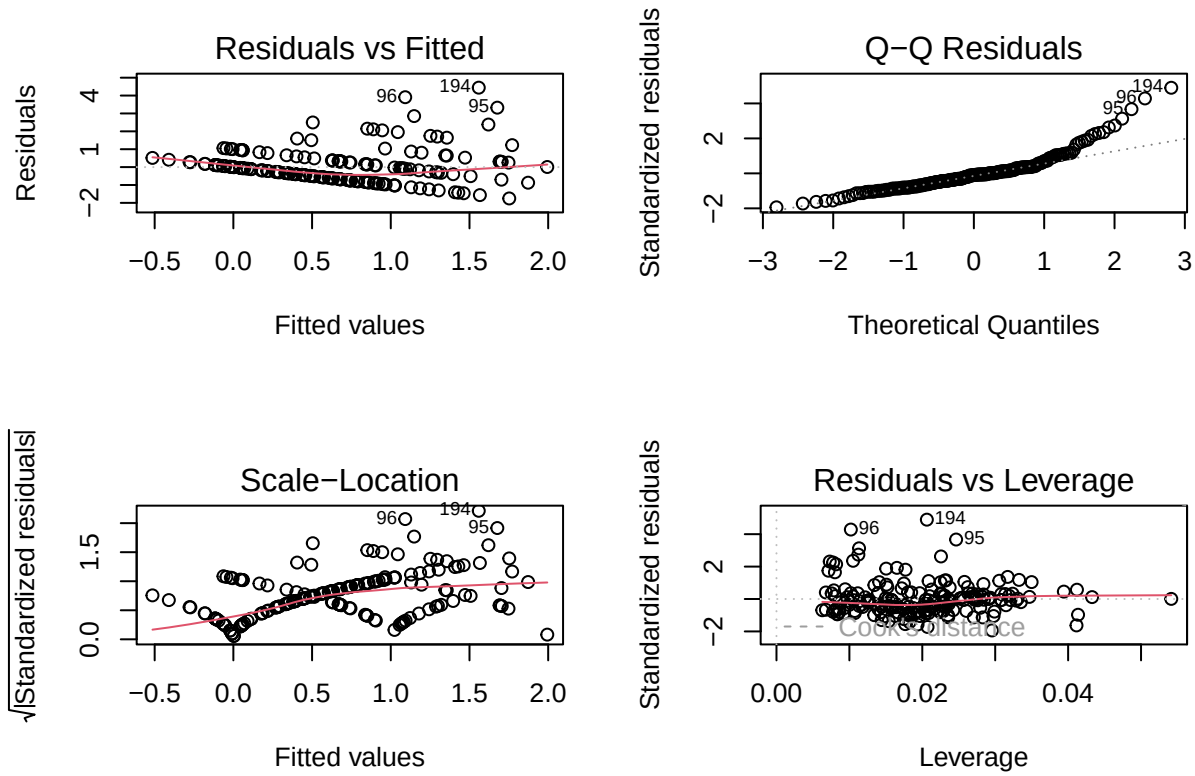
```r
simple_lm <- lm(num_awards ~ ., d)
summary(simple_lm)
```

```
##
## Call:
## lm(formula = num_awards ~ ., data = d)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -1.7526 -0.5680 -0.0862  0.2979  4.4411
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -2.536344   0.481442  -5.268 3.61e-07 ***
## prog         0.109815   0.095972   1.144    0.254
## math         0.056617   0.007130   7.941 1.53e-13 ***
## work        -0.002379   0.005807  -0.410    0.682
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.9168 on 196 degrees of freedom
## Multiple R-squared:  0.2533, Adjusted R-squared:  0.2418
## F-statistic: 22.16 on 3 and 196 DF,  p-value: 2.133e-12
```

```r
# What are the shortcomings of the simple regression?
plot(d$num_awards+runif(nrow(d),-0.2,0.2),
     predict(simple_lm, d),
     ylab = 'predicted no. awards',
     xlab = 'real no. awards',
     ylim = c(-1, 6))
```




```r
par(mfrow=c(2, 2))
plot(simple_lm)
```




```r
# Refer to the lectures what this call measures
anova(lm(num_awards ~ math + prog + work, d))
```

```
## Analysis of Variance Table
##
## Response: num_awards
##            Df  Sum Sq Mean Sq F value   Pr(>F)
## math        1  54.509  54.509 64.8496 7.69e-14 ***
## prog        1   1.222   1.222  1.4536   0.2294
## work        1   0.141   0.141  0.1679   0.6824
## Residuals 196 164.748   0.841
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

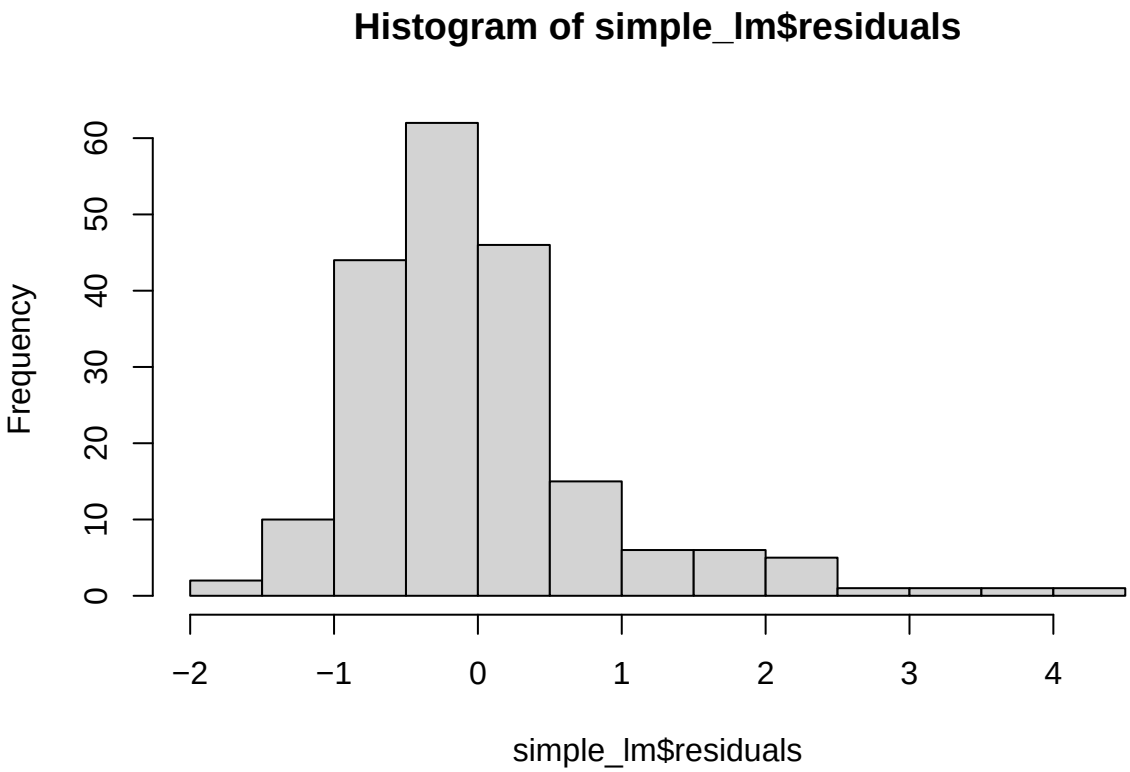
Add your verbal summary here (1p):

Q-Q plot shows us, that variables do not meet prerequisite of multivariate normality, since the right tail is significantly heavier. Residuals vs Fitted shows us that data are not linear, which violates another prerequisite. With this information, we cannot interpret p-value < 0.05 as we would with all prerequisites satisfied and reject H0. Also R-squared value is quite low, so the model is not fitting data well. Predictors prog and work are not significant.

Generalized linear model Now, the goal is to implement a generalized linear model that conceptually fits the given task. Do not transform the predictors yet, use them as they are, or omit them from the model. Once you obtain your model, interpret the effect of the *math* predictor on the outcome. How (according to your model) increasing a math score by one point affects the number of awards won?

Explain why the model overcomes the previous linear model, or justify that the generalized model is not needed. Compare the models theoretically as well as technically in terms of a proper quality measure. Note: The difference between the models cannot be statistically tested.

```r
#### add your code here ####
hist(simple_lm$residuals, plot = TRUE)
```




```r
glm.full <- glm(num_awards ~ ., family = poisson, data = d)
summary(glm.full)
```

```
##
## Call:
## glm(formula = num_awards ~ ., family = poisson, data = d)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -5.417786   0.707232  -7.661 1.85e-14 ***
## prog         0.118932   0.163815   0.726    0.468
## math         0.085209   0.009675   8.807  < 2e-16 ***
## work        -0.007073   0.008574  -0.825    0.409
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##     Null deviance: 287.67  on 199  degrees of freedom
## Residual deviance: 202.77  on 196  degrees of freedom
## AIC: 386.82
##
## Number of Fisher Scoring iterations: 6
```

Add your verbal summary here (1p):

Using histogram of residuals in linear model I chose poisson regression. R2 = 1-(RSS/TSS) = 0.295, which means that this model explains 4% more deviance. Predictors prog and work are not significant again.

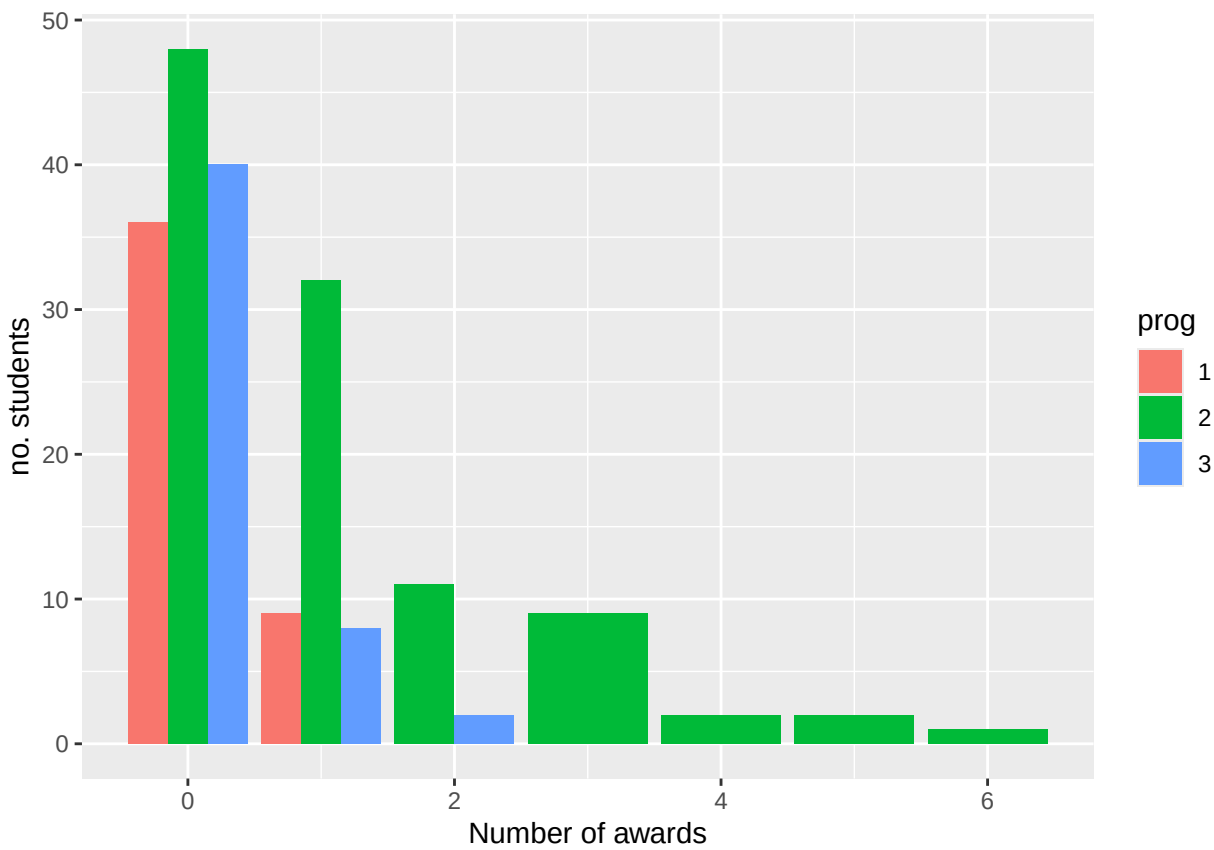
Feature transformations and final model *prog* and *work* did not prove to be effective predictors so far. Visualize these predictors as well as their relationship with the outcome variable. Based on the observations, propose suitable transformations for them (or, justify that they are truly uninformative for prediction of *num_awards*) and implement them into the best model found by now. Use the *compareGLM()* function to validate that your new GLMs indeed improved over the simple one.

```
typeof(d$prog)
```

```
## [1] "integer"
```

```
d %>%
  mutate(prog=as.factor(prog)) %>%
  group_by(num_awards, prog) %>%
  summarise(count=n()) %>%
  ggplot(aes(x = num_awards, y=count, fill = prog)) +
  geom_col(position="dodge") +
  xlab("Number of awards") +
  ylab("no. students")
```

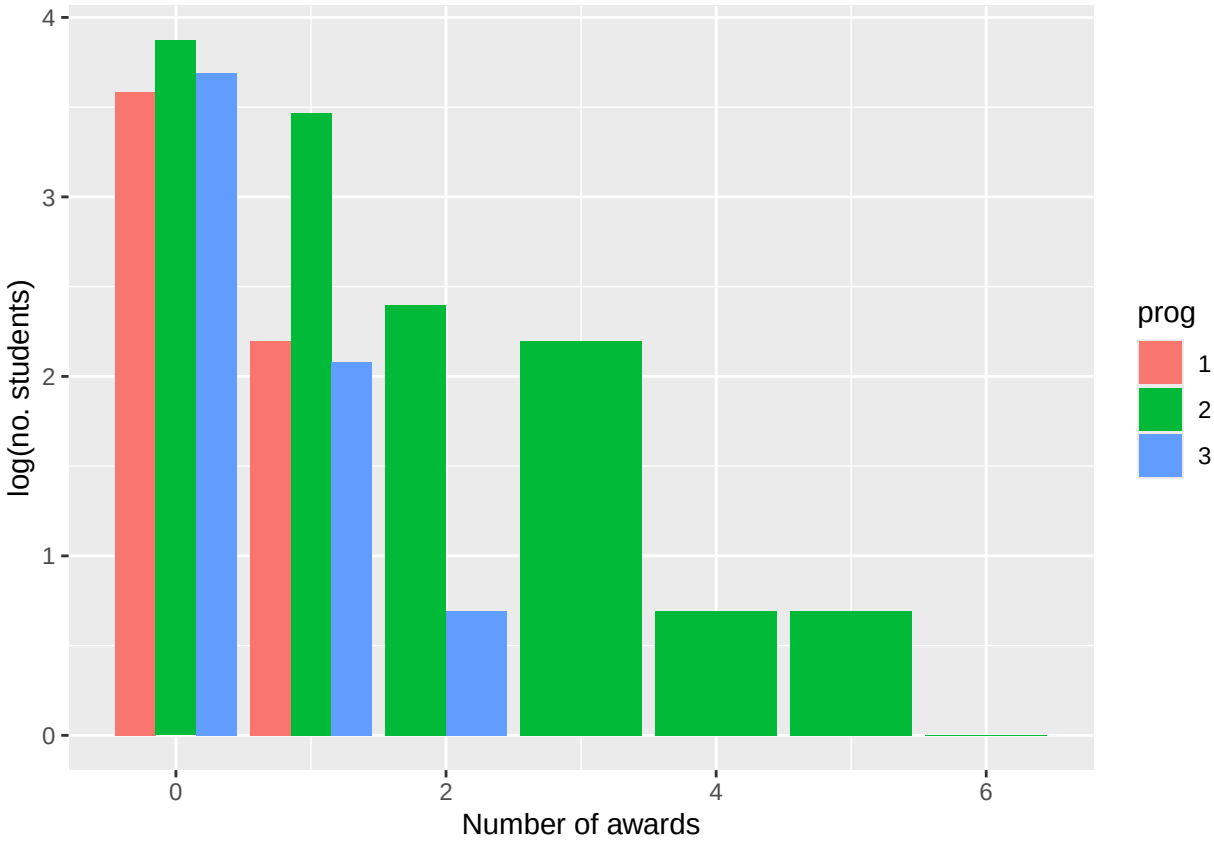
```
## `summarise()` has grouped output by 'num_awards'. You can override using the
## `.groups` argument.
```



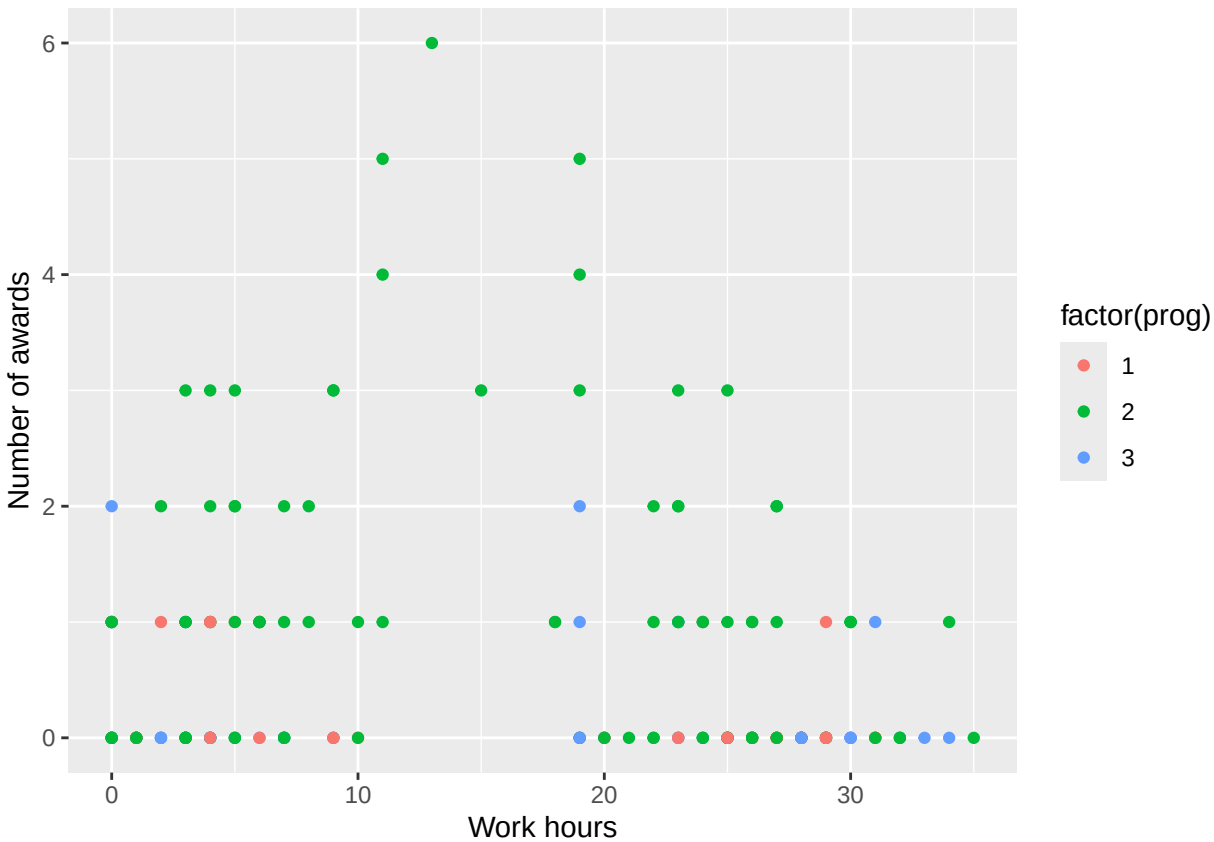

```
d %>%
  mutate(prog=as.factor(prog)) %>%
```

```r
  group_by(num_awards, prog) %>%
  summarise(count=n()) %>%
  ggplot(aes(x = num_awards, y=log(count), fill = prog)) +
  geom_col(position="dodge") +
  xlab("Number of awards") +
  ylab("log(no. students)")
```

```
## `summarise()` has grouped output by 'num_awards'. You can override using the
## `.groups` argument.
```




```r
d %>%
  ggplot(aes(x = work, y = num_awards, color = factor(prog))) +
  geom_point() +
  xlab("Work hours") +
  ylab("Number of awards")
```




```r
#### add your code here ####
glm.transform = glm(num_awards ~ math + log(prog) + poly(work, 2), family = poisson, data=d)
summary(glm.transform)
```

```
##
## Call:
## glm(formula = num_awards ~ math + log(prog) + poly(work, 2),
##     family = poisson, data = d)
##
## Coefficients:
##                 Estimate Std. Error z value Pr(>|z|)
## (Intercept)    -4.833256   0.621036  -7.783 7.11e-15 ***
## math            0.070535   0.009927   7.105 1.20e-12 ***
## log(prog)       0.376238   0.312742   1.203 0.228965
## poly(work, 2)1 -2.738720   1.650564  -1.659 0.097063 .
## poly(work, 2)2 -6.056328   1.575725  -3.844 0.000121 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##     Null deviance: 287.67  on 199  degrees of freedom
## Residual deviance: 184.91  on 195  degrees of freedom
## AIC: 370.96
##
## Number of Fisher Scoring iterations: 5
```

```r
compareGLM(glm.full, glm.transform)
```

```
## $Models
##   Formula
## 1 "num_awards ~ prog + math + work"
## 2 "num_awards ~ math + log(prog) + poly(work, 2)"
##
## $Fit.criteria
##   Rank Df.res   AIC  AICc   BIC McFadden Cox.and.Snell Nagelkerke   p.value
## 1    4    196 388.8 389.1 405.3   0.1831        0.3459     0.3837 1.344e-18
## 2    5    195 373.0 373.4 392.8   0.2216        0.4018     0.4456 1.245e-21
```

Add your verbal summary here (2p):

prediktor work vypadá jako polynomiální funkce. Vzhledem k 1 influxnímu bodu, volím polynom 2. řádu, u prog jsem na základě tvaru vyzkoušel log, který stále nedává signifikanci. CompareGLM ukazuje nižší AIC i BIC

Ablation study through cross-validation Recap all the models considered previously and evaluate them through cross-validation. You can start with the most simple null model and gradually add the previously discussed improvements. See their role in terms of MAE, RMSE and other commonly used criteria. The procedure outlined below proposes to work with *train* function from *caret* package, you can only add more models to evaluate and compare.

```r
train_control <- trainControl(method = "cv", number = 10)

# CV of the null lm model
train(x = data.frame(rep(1, nrow(d))),
      y = d$num_awards,
      method = "lm",
      trControl = train_control)$results
```

```
##   intercept     RMSE Rsquared       MAE    RMSESD RsquaredSD     MAESD
## 1      TRUE 1.010475      NaN 0.7816587 0.3066249         NA 0.1245651
```

```r
#### add your code here ####
train(num_awards ~ ., data = d, method = "lm", trControl = train_control)$results
```

```
##   intercept     RMSE  Rsquared      MAE    RMSESD RsquaredSD     MAESD
## 1      TRUE 0.910758 0.2812086 0.638298 0.1820012  0.1977839 0.0945401
```

```r
train(num_awards ~ ., data = d, family = "poisson", trControl = train_control)$results
```

```
## note: only 2 unique complexity parameters in default grid. Truncating the grid to 2 .
```

```
##   mtry      RMSE  Rsquared       MAE    RMSESD RsquaredSD     MAESD
## 1    2 0.9024631 0.3321223 0.6311565 0.2340409  0.2396144 0.1069404
## 2    3 0.9273067 0.3316284 0.6437788 0.2379982  0.2355717 0.1093779
```

```r
train(num_awards ~ math + log(prog) + poly(work, 2), data = d, family = "poisson", trControl = train_co
```

```
##   mtry      RMSE  Rsquared       MAE    RMSESD RsquaredSD     MAESD
## 1    2 0.8758209 0.3583817 0.6256041 0.2220144  0.1796478 0.1544486
## 2    3 0.8982798 0.3473549 0.6369643 0.2279854  0.1721201 0.1562390
## 3    4 0.9046617 0.3429426 0.6414677 0.2255885  0.1636894 0.1522634
```

Add your verbal summary here (1p):

Lineární model značně snižuje RMSE i MAE. Obecný GLM oba parametry nechá podobné. Transformací parametrů se pak dostáváme na nejnižší hodnoty, ne však výrazně. Zde je ale značně menší RMSESD a MAESD, což indikuje nejlepší model...